# TRIAGE-1 — Product Architecture & Roadmap

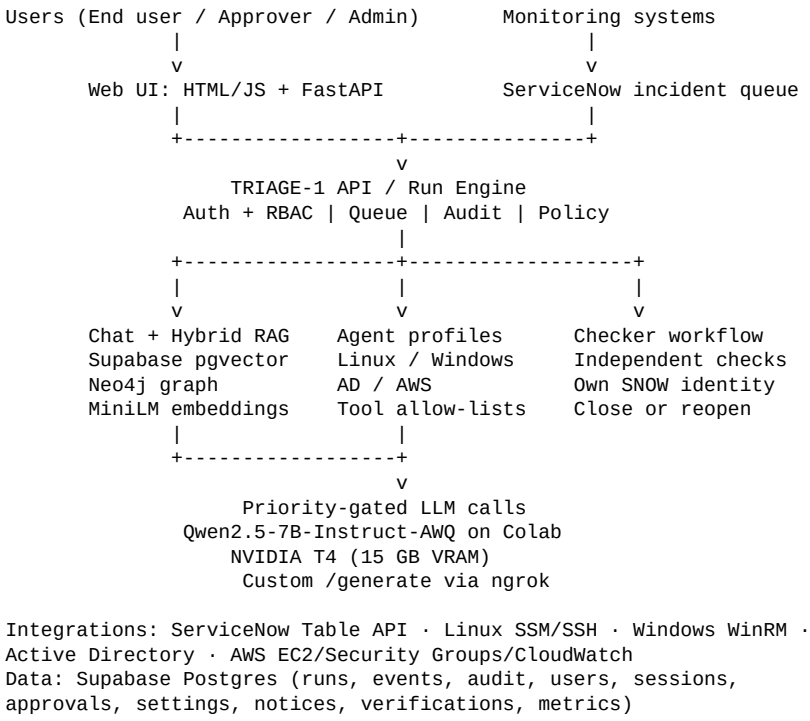
Concise system architecture, delivery status, and implementation roadmap

**Version:** Condensed from v0.6 · **Updated:** 18 Oct 2026 · **Owner:** Veda

## 1. Product purpose

TRIAGE-1 is an AI-assisted SRE triage agent that receives incidents from chat or monitoring, diagnoses infrastructure issues using runbooks and tools, proposes or performs permitted remediation, records activity in ServiceNow, and independently verifies resolutions before closing tickets.

## 2. Architecture at a glance




## 3. Technology stack

| Area | Current implementation |
|---|---|
| Web/API | FastAPI, static HTML/JavaScript, Tailwind CDN, SSE chat streaming |
| LLM | Qwen2.5-7B-Instruct-AWQ; one model on one Colab NVIDIA T4 (15 GB VRAM), exposed through ngrok |
| RAG | Supabase pgvector hybrid search (`hybrid_search` / `sre_runbooks`), Neo4j `Entity–RELATED_TO` graph, `all-MiniLM-L6-v2` embeddings |
| Agent framework | LangChain `create_agent`, LangGraph human-in-the-loop middleware and SQLite checkpoint |

| Area | Current implementation |
|---|---|
| Playbooks | `skills/<name>/SKILL.md`; code selects skills by keyword score and platform |
| Policy | `app/agent/policy.py`: fixed risk classes and tool permissions enforced in code |
| ITSM | ServiceNow Table API for incidents, notes, assignment, escalation, users and CIs |
| Infrastructure | Linux SSM/SSH, Windows WinRM, AD tools, AWS tools |
| Persistence | Supabase Postgres with RLS enabled and no public policies; SQLite is retained for the LangGraph checkpoint |
| Authentication | Login page, bcrypt password hashes, signed HttpOnly session cookie, roles: End user, Approver, Admin |
| Client chat history | Browser localStorage, namespaced by user ID |

**LLM constraint:** All model calls share one GPU and one priority gate: resume-after-approval → verify → triage → chat → ingest. Chat waits up to 90 seconds, then reports that the model is busy. Keep agent calls compact (target under 4K tokens); the model's 32K context limit is not practical on this GPU with KV-cache memory.

# 4. Main workflows

## A. Chat-based triage

User describes an issue; the app detects an incident request and filters off-topic turns from model context and ticket transcripts.

The agent identifies the host/CI, selects a platform-specific skill, gathers evidence and checks the relevant service.

The user can track ticket progress, SLA information and the outcome in chat.

Ticket creation is planned to use a confirmation overlay by default, with optional admin-enabled auto-create and a short undo window.

Chat can be disabled globally by an Admin; the user view then switches to **My tickets**.

## B. Monitoring-driven intake

- A poller checks the ServiceNow L1 assignment queue (currently every 15 seconds).
- Planned intake orders tickets by priority then age, reads the host from `cmdb_ci`, deduplicates repeated alerts, and supports an Admin kill switch.
- A circuit breaker disables auto mode after three consecutive failed runs.
- The auto-mode setting is persisted in Supabase and restored at startup.

## C. Diagnosis and remediation

- The deterministic router selects a profile for Linux, Windows, AD, or AWS/network work.
- Each profile contains its prompt, allowed tools, skills and policy; profiles share the same model rather than loading separate model copies.
- Read-only diagnosis is autonomous. Starting/restarting a stopped service may be autonomous only for approved critical services; non-critical service restoration requires approval.
- Configuration changes, patching, reboots, cleanup, data changes and identity changes require approval or human action. Data/identity changes are human-only.
- Critical-service autonomy requires preflight checks, one attempt, deterministic post-action verification, shadow mode before enablement, and a flap guard (three failures for the same service within an hour stops autonomy).

### D. Human approval

- Approvers see pending actions, ticket context, evidence collected, proposed action, risk level and a comment field.
- Approval/rejection is recorded in the approvals table and audit log; a ServiceNow work note records the decision and the agent resumes.
- Admins can also approve/reject and terminate stuck runs; their actions are audited under their own identity.
- Pending approvals survive restarts through persistent checkpointing and approval records.

### E. Independent resolution checker

- After TRIAGE-1 resolves an incident, a deterministic checker independently re-runs read-only verification using its own ServiceNow service account (`svc_triage_checker`).
- **Pass:** post one consolidated resolution report and close the incident.
- **Fail:** post evidence, reopen to In Progress, assign to the L2 escalation group, and propose P1 escalation for critical services. A second failed verification of the same ticket notifies L3.
- Checker modes: `off`, `shadow` (default; verify and note without changing state), and `on` (close/reopen).
- Service checks can be repeated after `VERIFY_SOAK_SECONDS` (default 60 seconds) to catch flapping. Disk and AD checks currently run once.
- Checker rules are currently in code and are intended to move into playbook `verify:` metadata in P6.

### F. Escalation and disruption notices

- ServiceNow derives priority from impact and urgency; TRIAGE-1 sets those fields rather than writing priority directly.
- High-criticality incidents start at P2. Failed verification, exhausted time budget, multiple CIs for one service, or approval timeout can propose P1 escalation.
- An approver reviews the P1 proposal, bridge link, draft notice and recipient list. Approval declares P1, posts the bridge link, sends the notice and audits the steps.
- MVP notices use AWS SNS; recipients are confirmed topic subscribers. Delivery failures must be visible and retried. The `notices` table prevents duplicate notices per incident/type.
- Notice text uses verified facts only; no invented root cause or ETA.

## 5. Identity, access and safety

### Roles

| Capability | End user | Approver | Admin |
|---|---|---|---|
| Chat and own-ticket progress | Yes | Yes | No; redirected to Admin |
| Pending approvals / approve / reject | No | Yes | Yes, audited |
| Agent Console | No | Yes | Read |
| Knowledge Management | No | No | Full access |
| User management, config, audit, health | No | No | Yes |
| Terminate stuck runs | No | No | Yes |

- User identity is based on lower-cased ServiceNow email; the user record stores ServiceNow username and `sys_id`.
- Access control is enforced server-side for pages and APIs; forbidden direct URLs must return 403.
- Backend uses the Supabase **secret** key; RLS is enabled on every table with no public policies.

- Risk rules and permissions are enforced in code, not solely in prompts. Unknown tools default to L2/human approval; L3 actions notify/escalate only.
- RAG queries are redacted before logging; use hashed user IDs and retention limits. Playbook content is a security boundary and must be validated and permission-controlled.

## 6. Data and observability

Supabase Postgres stores users, sessions, runs, events, audit records, approvals, settings, notices, verification outcomes, LLM usage and RAG/chat events. Key measurements include:

- Ticket volume, resolution/escalation/error rates, agent MTTR and approval wait time.
- Tool failures, steps/nudges per run, checker pass/reopen rate and skill coverage.
- Input/output tokens, model latency, queue wait time and Colab uptime.
- RAG retrieval scores, cited vs phantom documents, KB gaps, top/unused documents, per-stage latency and index health.
- Chat-created tickets, off-topic rate, cancellations/undo and wrap-up outcomes.

RAG observability still needs `hybrid_search` to return scores. Do not retain raw user queries without redaction.

## 7. Playbooks and token efficiency (planned P6)

- Move playbooks to a dedicated ServiceNow Knowledge Base (`TRIAGE-1 Playbooks`) so approved edits can change behaviour without deployment.
- Fetch only published, active articles; parse and validate metadata; cache in memory with a TTL; fall back to local `SKILL.md` files if ServiceNow is unavailable.
- Record article number and version/hash on each run.
- KB migration alone does **not** reduce tokens. Main savings are a step engine that sends only the current step and latest tool result, concise playbooks/prompts, narrow tool descriptions, structured/truncated tool results and deterministic checks.

## 8. Current delivery status (v0.6, 18 Oct 2026)

| Phase | Scope | Status |
|---|---|---|
| P0 | Metrics, token logging, priority gate, skill matching | **Done and verified** |
| P1 | Login, roles, sessions, role-aware UI and per-user chat storage | **Done** |
| P1a | Persistent auto intake, queue order, breaker, chat switch | **In progress** — host-from-CMDB, alert dedupe and My tickets view remain |
| P2 | Approver workspace and restart-safe approvals | **Done and tested** |
| P2a | Critical-service autonomous restore and safety guardrails | **In progress** — CMDB context, `nginx -t`, per-service enablement and deterministic post-fix checks remain |
| P2b | P1 proposal, approval flow, notice tracking/retry | **In progress** — SNS delivery retest pending; per-incident bridge and follow-up/resolved notices not built |
| P3 | Chat ticket overlay, auto-create and undo | **Not started** |
| P4 | Admin console and RAG observability | **Not started** |

| Phase | Scope | Status |
|---|---|---|
| P5 | Independent checker and close/reopen workflow | **In progress** — close by `svc_triage_checker` verified in `on` mode; fresh-ticket resolution report and flap/reopen test pending |
| P6 | ServiceNow KB playbooks and step engine | **Not started** |
| P7 | Named profiles, hardened queue/orchestration, SSO and deployment hardening | **Not started** |

**Immediate next step:** finish P5 live verification (fresh-ticket resolution report and flap/reopen test), then proceed to P6. Also retest SNS notice delivery.

# 9. Open decisions / inputs needed

- **Q13:** Example monitoring-created incident fields: short description, description, caller, `cmdb_ci`, assignment group and priority.
- **Q15:** Critical-service list and the CMDB field/value that defines criticality.
- **Q16:** Confirm autonomous scope is limited to restarting a stopped critical service.
- **Q17:** Confirm P2-to-P1 time budget and triggers (current default is 15 minutes).
- **Q18:** Keep both the agent step-by-step report and checker report, or shorten the agent report?
- **Q19:** Later decision: verify tickets resolved by humans and ask callers to confirm resolution?
- Also confirm the ServiceNow auto-close rule, SNS delivery, standing bridge link and real SLA values.

# 10. Key constraints and known risks

- One shared T4 model limits throughput; queue priority and the semaphore prevent collisions.
- The checker must never close a ticket when it cannot verify the host or state.
- Critical-service autonomy can cause disruption; use code-enforced risk classes, preflight checks, one attempt, deterministic verification, flap protection, shadow mode and an Admin kill switch.
- A published playbook can change agent behaviour; restrict KB publishing, validate tool names and keep `policy.py` as the hard boundary.
- Prevent duplicate tickets/notices; audit actions, redact sensitive data, and preserve restart-safe state.
- Check ServiceNow ACLs and auto-close rules before relying on close/reopen behaviour.

# 11. Development handoff rules

Do not rewrite working core functionality; extend it.

Complete one phase and meet its exit criteria before starting the next.

The owner applies code manually. Specify exact file paths; provide complete new/short files or current/replacement snippets for long files.

Never rely on prompt text alone for safety or permissions.

After each phase, update status, open questions, progress/changelog and handoff notes.

Before code changes, request a fresh repository dump; the existing dump may lag behind the repository.